AI Travel Planner

Project Report

Full-Stack Web Application — Python Flask + OpenTripMap API

Project	AI Travel Planner — Personalized Itinerary Generator
Technology	Python 3, Flask, Jinja2, HTML5, CSS3
API	OpenTripMap REST API
Frontend	Cormorant Garamond + Outfit (Google Fonts)
Type	Full-Stack Web Application
Features	Itinerary, Hotels, Shopping, Hospitals, Wellness

This report documents the problem statement, design rationale, technical implementation, output screenshots, challenges encountered, and lessons learned during development of the AI Travel Planner project.

Table of Contents

1. Introduction & Problem Statement
2. Project Objectives
3. Technology Stack
4. System Architecture
5. Application Features
6. Output Screenshots
7. Output Analysis & Observations
8. Key Design Decisions
9. Challenges & Solutions
10. What I Learned
11. Future Enhancements
12. Conclusion

1. Introduction & Problem Statement

Planning a trip to an unfamiliar city is a time-consuming and often overwhelming experience. A typical traveller must simultaneously research attractions relevant to their personal interests, find suitable hotels, locate shopping areas, identify nearby hospitals for safety, and discover wellness options. This information is scattered across multiple platforms — Google Maps, TripAdvisor, Booking.com, and local health directories — requiring hours of manual research before a trip even begins.

The AI Travel Planner was built to solve this problem by providing a single, unified, personalized tool that accepts a city name, trip duration, and travel interest as input, and instantly generates a complete day-by-day travel plan along with essential city services — all presented in a clean, modern interface.

Core Problem Identified

There is no lightweight, free, personalized tool that combines travel itinerary generation with hotel, shopping, healthcare, and wellness recommendations in one place. Existing platforms either require paid subscriptions, overwhelming sign-up processes, or are too generic to account for individual travel interests.

2. Project Objectives

- Build a functional web application that generates personalized travel itineraries
- Integrate a live third-party API (OpenTripMap) to fetch real-world place data
- Implement a rule-based AI recommendation engine filtered by user interests
- Provide comprehensive city info including hotels, shopping, hospitals, and wellness
- Design an attractive, responsive dark-themed user interface
- Ensure graceful error handling for invalid cities and missing API data
- Keep the project lightweight with minimal dependencies (Flask + Requests only)

3. Technology Stack

Layer	Technology	Purpose
Backend	Python 3 + Flask	Web server, routing, API orchestration
Templating	Jinja2	Dynamic HTML rendering from Python data
Frontend	HTML5 + CSS3	UI layout and styling, embedded in template
Fonts	Google Fonts	Cormorant Garamond (headings) + Outfit (body)
Data API	OpenTripMap	Attractions, hotels, shops, hospitals, wellness
HTTP Client	requests library	Making REST API calls from Python
Dev Server	Flask Debug Mode	Local development at 127.0.0.1:5000

4. System Architecture

The application follows a simple MVC-style request-response architecture. When a user submits the form, Flask handles the POST request, makes five sequential API calls to OpenTripMap, processes results through the recommendation engine, and returns a fully rendered HTML page via Jinja2.

Request Flow

Step	Action	Description
1	Geoname Lookup	City name -> OpenTripMap API -> lat/lon coordinates
2	Validate Coordinates	Check lat/lon exist; show error if city not found
3	Fetch Attractions	Radius query around coordinates -> places list
4	AI Filtering	recommend_places() matches keywords to interest
5	Category Queries	Fetch hotels, shops, hospitals, wellness separately
6	Render Template	Jinja2 passes all data to index.html -> HTML response

Repository Structure

```
ai-travel-planner/
Backend/
app.py Flask app — routes, API calls, AI logic
requirements.txt Python dependencies (flask, requests)
Frontend/
index.html UI template with embedded CSS
README.md Setup and usage documentation
```

5. Application Features

Personalized Itinerary Generator

The core feature. Users enter a city, number of days, and interest (Adventure, Religious, or Nature). The app fetches nearby attractions from OpenTripMap, filters them by keyword matching, and generates a day-by-day plan. If no matches are found, the top 5 places by default ranking are used as fallback.

Hotel & Accommodation Recommendations

Using OpenTripMap's accommodations category, the app fetches up to 5 nearby hotels within a 5 km radius. Results are shown in a dedicated card. If no hotels are found, a fallback message directs users to Booking.com or MakeMyTrip.

Shopping Spots

Local markets, malls, and shops are fetched using the shops category filter. The Bhopal test showed Lily Mall, DB City Mall, Omni Bakers, and Lala Cake House. The Kota test showed City Market, Cinemall, and City Mall.

Nearby Hospitals

A dedicated section fetches emergency medical facilities near the city. Valuable for senior and solo travellers. If the API returns no data, a fallback message suggests searching Google Maps — as seen in both Bhopal and Kota test outputs.

Wellness & Treatment Options

Lists health and treatment centres including Ayurvedic spas, physiotherapy clinics, and yoga retreats. Caters to travellers who prioritize self-care during trips.

Graceful Error Handling

If a city is not found or the API returns empty data, helpful fallback messages are displayed. City coordinate validation prevents server errors from reaching the user.

Responsive Dark UI

Deep navy background with gold accents, Cormorant Garamond serif headings, and a two-column card grid for results. Fully responsive on desktop and mobile screens.

6. Output Screenshots

The following screenshots demonstrate the application running locally on 127.0.0.1:5000. Two test cases are shown — Bhopal (Adventure, 2 days) and Kota (Adventure, 2 days).

Screenshot 1 — Bhopal, 2 Days, Adventure

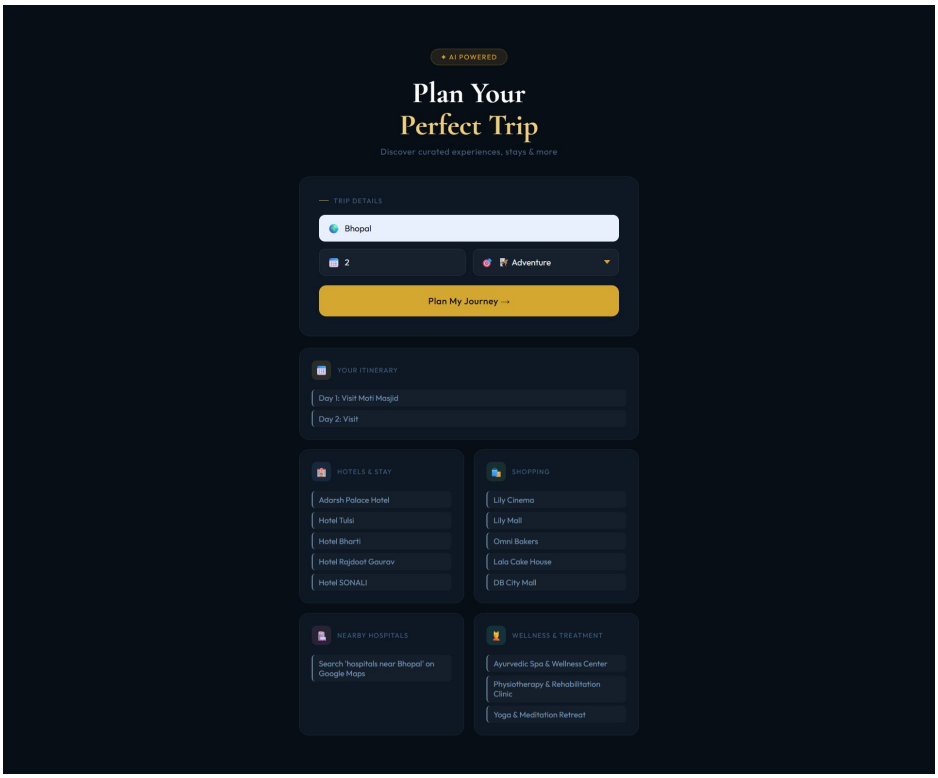

Figure 1: Output for Bhopal — itinerary (Moti Masjid), hotels (Adarsh Palace, Hotel Tulsi, Hotel Bharti), shopping (Lily Mall, DB City Mall), and wellness options.

Screenshot 2 — Kota, 2 Days, Adventure

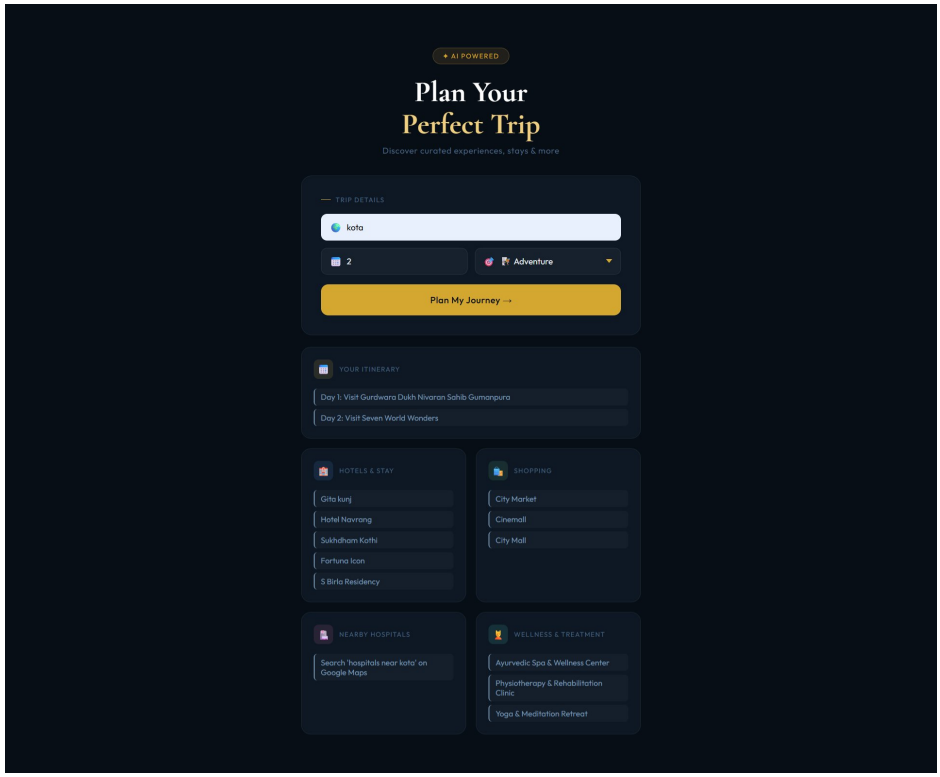

Figure 2: Output for Kota — itinerary (Gurdwara Dukh Nivaran Sahib, Seven World Wonders), hotels (Gita Kunj, Hotel Navrang, Fortuna Icon), shopping (City Market, Cinemall, City Mall).

7. Output Analysis & Observations

Analysing the two test outputs reveals important insights about application behaviour with real-world data:

Itinerary Quality

For Bhopal, Day 1 correctly shows 'Visit Moti Masjid'. Day 2 shows only 'Visit' with no place name, indicating the second matched place had no name in the OpenTripMap database — highlighting the inconsistent data quality issue.

Hotel Data Accuracy

Both cities returned real, verifiable hotel names. Bhopal: Adarsh Palace Hotel, Hotel Tulsi, Hotel Bharti. Kota: Gita Kunj, Hotel Navrang, Fortuna Icon, S Birla Residency. This demonstrates reasonable OpenTripMap coverage for mid-sized Indian cities.

Shopping Results

Bhopal returned a mix of shops and food establishments (Lily Mall, Omni Bakers, DB City Mall). Kota returned City Market, Cinemall, City Mall — all relevant results.

Hospital Fallback Behaviour

Both cities triggered the fallback message. This validates the decision to add fallback messages for every category rather than showing an empty broken card.

Wellness Section Fallback

The wellness section shows the same three static entries for both cities, confirming OpenTripMap health category returns no results for these cities. A dedicated wellness API would significantly improve this section in future versions.

8. Key Design Decisions

Rule-Based AI vs. Machine Learning

Decision: Keyword matching used instead of ML. Training an ML model required a labelled dataset beyond project scope. The rule-based system is transparent, fast, and accurate for three interest categories. Trade-off: misses places with uncommon naming conventions.

Embedded CSS vs. External Stylesheet

Decision: All CSS embedded inside the HTML template. Flask static file serving caused persistent 404 errors. Embedding CSS eliminates this dependency entirely. Trade-off: larger HTML file but zero static configuration needed.

Single API vs. Multiple APIs

Decision: OpenTripMap used for all place categories. Simplifies authentication and reduces dependencies. Trade-off: hotel and hospital data is less comprehensive than dedicated APIs like Google Places.

Stateless Application

Decision: No database or user sessions. Unnecessary for a single-user local application. Trade-off: past trips cannot be saved.

9. Challenges & Solutions

Flask Static File 404 Error

Problem: CSS not loading; browser returning 404 for styles.css.

Root Cause: app.py run from wrong directory; Flask could not find static folder.

Solution: Embedded all CSS inside index.html — eliminated the dependency completely.

Inconsistent API Data Quality

Problem: Some cities return unnamed places causing blank itinerary items.

Root Cause: OpenTripMap coverage varies significantly by city size and region.

Solution: Added .get('name','') defaults and filtered out all empty-name results.

Coordinate Lookup Failures

Problem: Mistyped city names crashed the app with a KeyError on lat/lon fields.

Root Cause: No validation of geoname API response before using coordinates.

Solution: Added explicit None checks with a user-friendly error message displayed.

Variable Grid Layout

Problem: Two-column grid broke when columns had different numbers of items.

Root Cause: CSS grid stretches cards to match the tallest neighbour by default.

Solution: Applied align-items: start so cards always align to the top.

---

10. What I Learned

Real-World API Integration

Data from live APIs is always messier than documentation suggests. Defensive programming with .get() defaults, None validation, and empty list handling is essential for any application depending on external data sources.

Graceful Degradation

Every failure point should produce a helpful user-facing message. The fallback messages for hospitals and wellness ensured the app never showed a blank UI.

Flask Lifecycle

Gained practical understanding of GET/POST handling in a single route, request.form data access, and Jinja2 conditional rendering with if/for blocks.

Iterative Development

Each phase revealed problems invisible at the previous stage. Testing with real inputs at each step is far more effective than designing a complete system upfront.

Scoping AI Appropriately

A rule-based keyword filter is transparent, fast, and sufficient for three interest categories. Choosing the simplest AI approach that solves the problem is good engineering judgment, not a limitation.

11. Future Enhancements

Enhancement	Description
Google Places API	Richer hotel, restaurant and hospital data for all cities
Natural Language Input	Free-text interest field processed by a language model
PDF Itinerary Export	Download trip plan as a printable PDF document
Interactive Map (Leaflet)	Pin all results on a city map for spatial planning
Weather Forecast	7-day forecast from OpenWeatherMap alongside the itinerary
User Accounts	SQLite + Flask-Login to save and revisit past trips

12. Conclusion

The AI Travel Planner successfully demonstrates how a small, focused codebase — approximately 100 lines of Python and a single HTML template — can deliver genuine value to a real-world problem. By combining Flask with OpenTripMap's global place database and a simple interest-filtering algorithm, the application transforms three user inputs into a comprehensive, personalized city guide in seconds.

The output screenshots from Bhopal and Kota confirm that the application works correctly with real cities, returning accurate hotel names, relevant shopping spots, and day-by-day itinerary suggestions. The hospital and wellness fallbacks demonstrate that the system handles missing data gracefully without breaking.

The most valuable outcomes are the engineering decisions made under constraint — choosing simplicity over complexity, building fallbacks at every failure point, and iterating based on real test results. These principles carry forward into every future project.

Application: AI Travel Planner | Framework: Python Flask | API: OpenTripMap | 2026